

기초 알고리즘

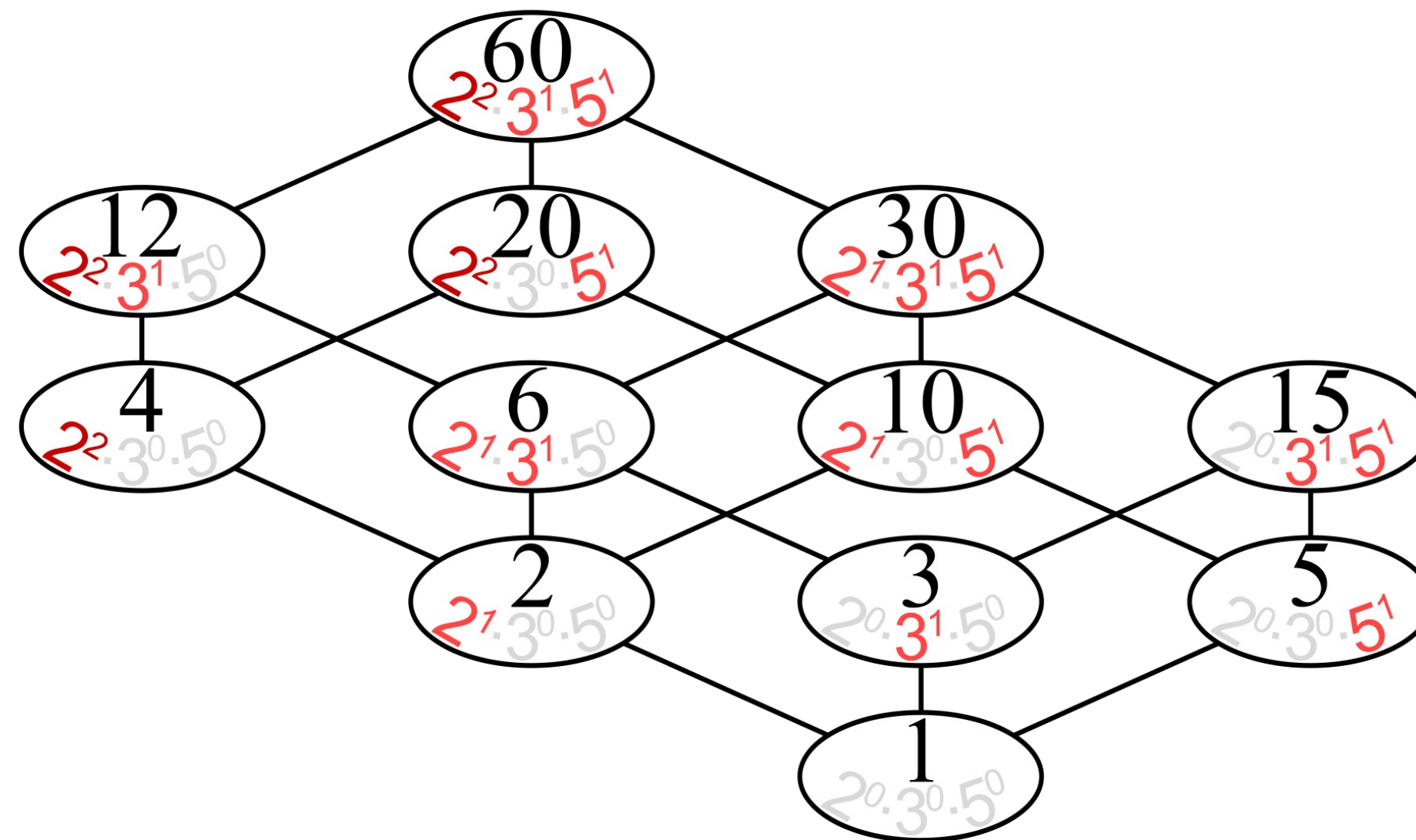
알고리즘 분석을 위한 수학 (2)

유클리드 호제법



약수(Divisor)

- 어떠한 수를 나누어 떨어지게 하는 수
- 나누어 떨어진다 → 나머지가 0





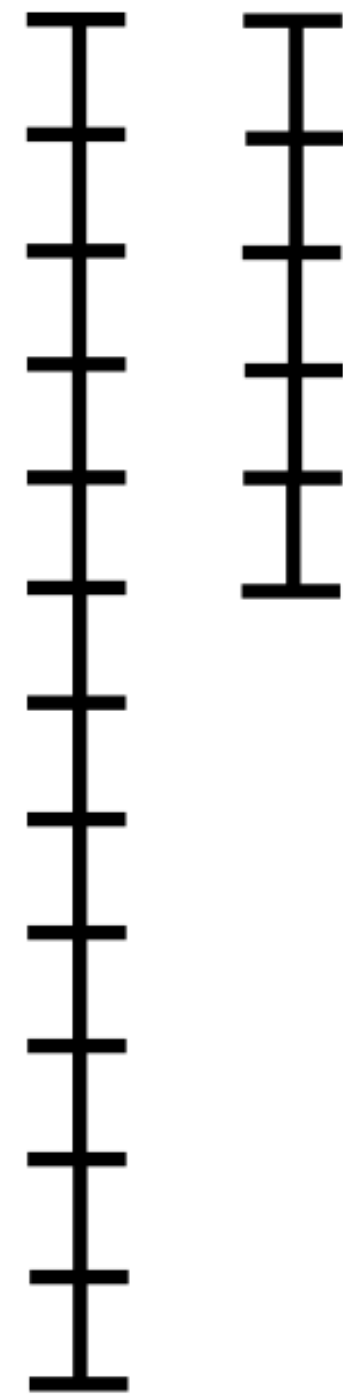
약수(Divisor)

- N 의 약수의 개수를 구하는 방법
 - 1부터 N 이하의 정수로 N 을 나누어서, 나머지가 0이 되는 수의 개수를 찾음
 - 시간 복잡도 : $O(N)$
 - 1부터 $\sqrt{N}$ 이하의 정수로 N 을 나누어서, 나머지가 0이 되는 수의 개수를 찾고, 그 개수에 2를 곱함
 N 이 제곱수면, 그 수에 1을 뺀
 - 시간 복잡도: $O(\sqrt{N})$
 - 제곱수 : 어떠한 정수의 제곱이 되는 정수



유클리드 호제법(Euclidean Algorithm)

- 호제법이란 말은 두 수가 서로(互) 상대방 수를 나누어(除) 원하는 수를 구하는 것을 의미
- 두 자연수 a, b 에 대해서($a > b$) a 를 b 로 나눈 나머지를 r 이라 하면, a 와 b 의 최대공약수는 b 와 r 의 최대공약수와 동일
- 이 성질에 따라, b 를 r 로 나눈 나머지 r' 를 구하고, 다시 r 을 r' 로 나눈 나머지를 구하는 과정을 반복하여, 나머지가 0이 되었을 때 나누는 수가 a 와 b 의 최대공약수





예시

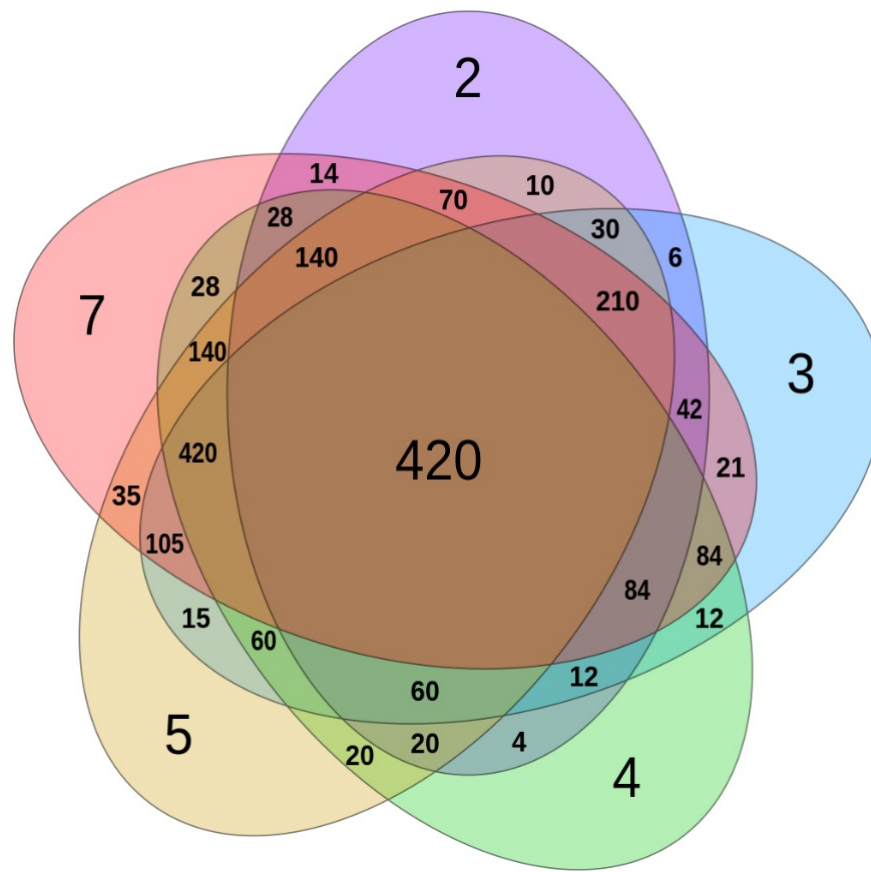
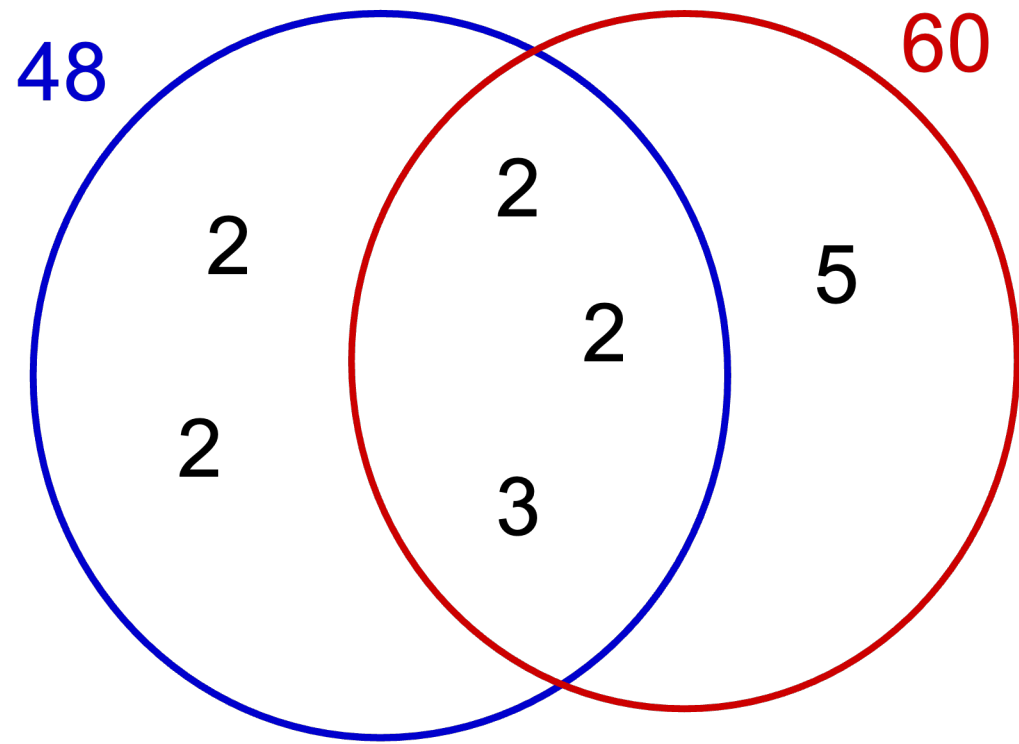
```
def GCD(a, b):  
    while b:  
        a, b = b, a % b  
    return a
```

1071과 1029의 최대공약수를 구해보자.

- 1071은 1029로 나누어 떨어지지 않기 때문에,
1071을 1029로 나눈 나머지를 계산
→ 42
- 1029는 42로 나누어 떨어지지 않기 때문에,
1029를 42로 나눈 나머지를 계산
→ 21
- 42는 21로 나누어 떨어지므로,
1071과 1029의 최대공약수는 21

코드로 이를 구현한다면?

GCD와 LCM



- GCD(Greatest Common Divisor = 최대공약수)
- LCM(Least Common Multiple = 최소공배수)
 - $\text{LCM}(a, b) = a \times b \div \text{GCD}(a, b)$
 - 어떠한 두 수의 곱은, 그 두 수의 최대공약수와 최소공배수의 곱과 같다.



표준 라이브러리 사용

```
import math

a = 12
b = 18

print("GCD:", math.gcd(a, b))
print("LCM:", math.lcm(a, b))
```

Python에서는 math 모듈의 gcd와 lcm 함수 사용

→ gcd는 python 3.5, lcm은 python 3.9 이상 지원

코딩 테스트 사이트에서 제공하는

언어의 버전에 대해 사전 체크 필요



140 48

예를 들어서, 다음과 같이 두 숫자가 있다고 가정해보자



140 48

큰 수를 작은 수로 나눴을 때의 나머지를 구해보면?



140 48

나머지는 44

나머지가 있다는 말은,

다시 말하자면 140가 48로 나눠 떨어지지 않는다는 말



140 48
44

이제 44와 48이라는 숫자가 있을 때,
큰 수 48에서 작은 수 44를 나눈 나머지를 구해보면?



140	48
44	4

이번에는 4를 얻을 수 있음



140 48

44 4

다시 44를 4로 나누면?



140 48

44 4

0

44를 4로 나눈 나머지는 0

즉, 나눠 떨어진다는 것



140 48

44 4

놀랍게도, 이렇게 얻어낸 4라는 숫자는

140과 48의 최대공약수



그렇다면, 이게 왜 성립하는지를 정리해보면?



$$140 = 48 \times 2 + 44$$

140을 48로 나눈 나머지가 44이므로,

다음과 같이 정리할 수 있음



$$140 = (44 \times 1 + 4) \times 2 + 44$$

그리고, 48을 44로 나눈 나머지가 4이므로,

식을 다음과 같이 바꿀 수 있음



$$140 = (\underline{44} \times 1 + \underline{4}) \times 2 + \underline{44}$$

마지막으로 44는 4로 나뉘지므로,
각각의 항이 모두 4로 나뉘지는 것을 확인할 수 있음



$$140 = (\underline{44} \times 1 + \underline{4}) \times 2 + \underline{44}$$

따라서, 계속 나누는 것을 반복하면서

나머지가 0이 되는 순간의 값을 최대공약수로 볼 수 있음

에라토스테네스의 체



소수란?

prime	composite
•• 2	
••• 3	
	4 ••
••••• 5	
	6 •••
••••••• 7	
	8 ••••
	9 •••
	10 •••••
•••••••••• 11	
	12 ••••

- 소수(Prime Number)란 1과 자기 자신으로만 나누어지는 수를 의미 → 약수가 2개

- 합성수(Composite Number)란 약수의 개수가 3개 이상인 수

- 단, 1은 소수도 아니고 합성수도 아님



소수 판별법(Primality Test)

- N 의 약수의 개수를 세는 방법에는 크게 두 가지
 - 1부터 N 까지의 수로 N 을 나눔
 - $O(N)$ 의 시간 복잡도
- 1부터 $\sqrt{N}$ 까지의 수로 N 을 나눔
- $O(\sqrt{N})$ 의 시간 복잡도





소수 판별법(Primality Test)

- 두 방법은 단 하나의 수에 대해서만 소수를 판별
- 만약 1부터 N 까지의 수들 중 소수를 판별하게 된다면,
기존의 시간 복잡도에 $\times N$
→ 더 좋은 방법은 없을까?





에라토스테네스의 체

- 에라토스테네스의 체는 이름 그대로 체로 걸러낸다고 생각하면 된다.



- 에라토스테네스의 체로 소수가 아닌 수들을 걸러내보자.



에라토스테네스의 체

	2	3	4	5	6	7	8	9	10	Prime numbers
11	12	13	14	15	16	17	18	19	20	
21	22	23	24	25	26	27	28	29	30	
31	32	33	34	35	36	37	38	39	40	
41	42	43	44	45	46	47	48	49	50	
51	52	53	54	55	56	57	58	59	60	
61	62	63	64	65	66	67	68	69	70	
71	72	73	74	75	76	77	78	79	80	
81	82	83	84	85	86	87	88	89	90	
91	92	93	94	95	96	97	98	99	100	
101	102	103	104	105	106	107	108	109	110	
111	112	113	114	115	116	117	118	119	120	

동작 순서

1. 2부터 N 까지 모든 정수를 적는다.
2. 아직 지우지 않은 소수 중 가장 작은 소수를 찾는다. 이 수를 P 라고 한다.
3. 아직 지우지 않은 수들 중, P 의 배수를 크기 순서대로 지운다.
4. 아직 모든 수를 지우지 않았다면, 다시 2번 단계로 간다.

$O(N \log(\log N))$ 의 시간 복잡도



에라토스테네스의 체

```
for i in range(2, N + 1):  
    # 아직 지우지 않은 수 중 가장 작은 소수를 찾음  
    if is_prime[i] :  
        for j in range(i * 2, N + 1, i):  
            # j가 소수가 아님을 is_prime 배열에 체크  
            if is_prime[j] : is_prime[j] = False
```

bool 자료형 배열인 *is_prime* 안의 모든 요소들에 대하여, 처음에 *True*로 초기화

1은 소수가 아니므로, *i*값은 2부터 시작

*j*는 *i*만큼 계속 커지도록 하여, *i*를 제외한 *N* 이하의 모든 *i*의 배수를 빠르게 체크

이후, *is_prime[i] = True*인 모든 *i*의 값은 소수가 됨



37

2

3

4

5

6

조금만 더 생각해 보면, 결국 소수인지 확인하기 위해서는
제곱근보다 작은 **소수**로만 나눠보면 된다는 것을 알 수 있음



에라토스테네스의 체

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40

이 방법을 활용해 소수를 더 빠르게 구할 수 있는
알고리즘이 있음



에라토스테네스의 체

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40

10이 소수가 아니라는 건 알고 있으니, 제외하고 시작



에라토스테네스의 체

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40

2는 소수이니 소수로 체크



에라토스테네스의 체

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40

당연히 2의 배수 (4, 6, 8, ...) 는 소수가 아니므로,
모든 2의 배수를 제외



에라토스테네스의 체

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40

이런 과정을 반복하면서, 어떤 수가 자기 차례가 될 때까지
선택되지 않았다면 소수로 보고, 그 수의 배수를 모두 제거



에라토스테네스의 체

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40

이 과정이 에라토스테네스의 체



에라토스테네스의 체

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40

3은 소수로 볼 수 있으니, 3의 배수를 모두 제거해주세요.



에라토스테네스의 체

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40

다음 소수는 5가 되니, 5의 배수를 제거



에라토스테네스의 체

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40

이후 남은 수들은 더 지울 수 없음



에라토스테네스의 체

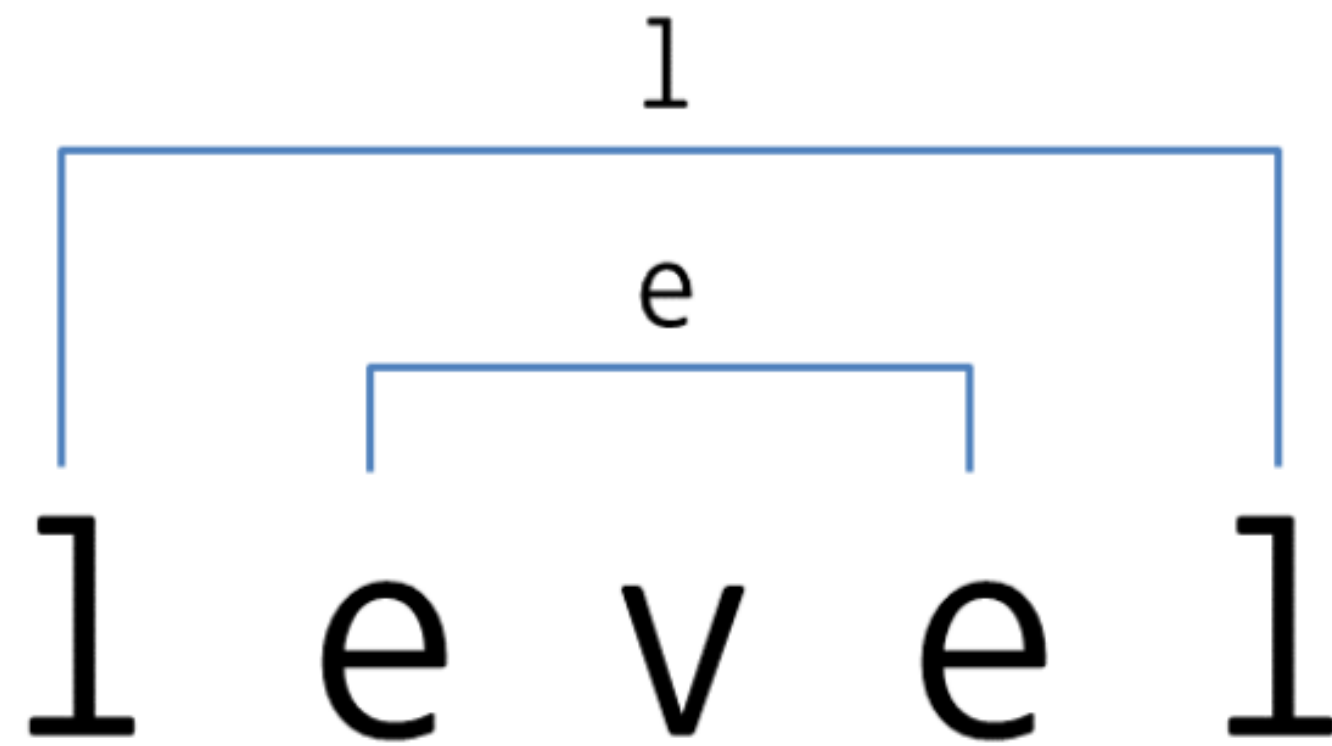
1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40

따라서 모든 남은 수들이 소수

문자열

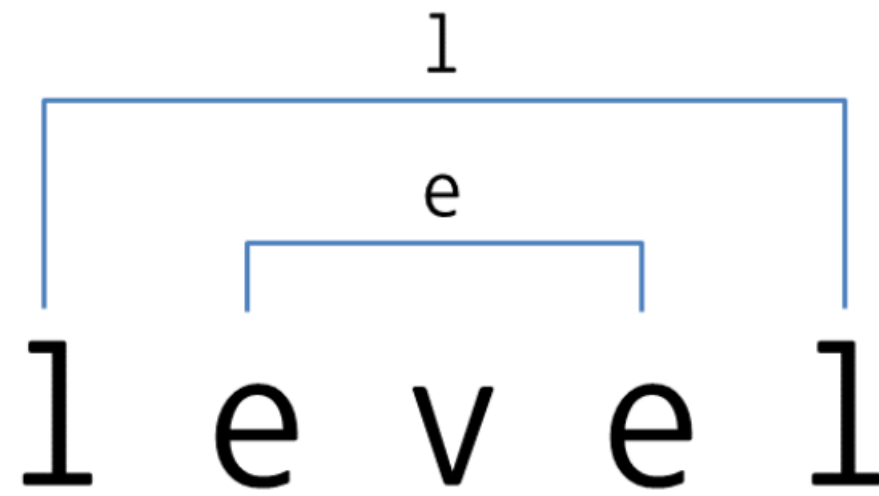
회문(Palindrome)

- 앞뒤 방향으로 볼 때, 같은 순서의 문자로 구성된 문자열을 의미
- 회문 예시: "소주 만 병만 주소", "Madam, I'm Adam", "1234321"
- 회문을 판단하려면?





회문(Palindrome)



```
def is_palindrome(s):  
    for i in range(len(s)):  
        if s[i] != s[len(s) - i - 1]:  
            return False  
    return True
```

```
def is_palindrome(s):  
    return s == s[::-1]
```

올바른 괄호 문자열(Valid Parenthesis String, VPS)

1. 빈 문자열은 올바른 괄호 문자열이다.
2. S 가 올바른 괄호 문자열이라면, (S) 도 올바른 괄호 문자열이다.
3. S, T 가 올바른 괄호 문자열이라면, ST 도 올바른 괄호 문자열이다.

S 가 입력될 때, S 가 올바른 괄호 문자열인지 판단하여라.

- 보통은 스택(Stack)을 사용해서 해결하는 문제
- ')'가 입력될 때마다, 스택에 있는 '('를 하나씩 지움
 - 이때, 스택(top)이 비어 있거나, '('가 없으면 올바른 괄호 문자열이 아님
- 모든 문자열을 순회한 뒤, 스택이 비어 있으면 올바른 괄호 문자열이고, 비어 있지 않으면 올바르지 않은 괄호 문자열임
- 이를 코드로 구현하면?



스택(Stack)이란?

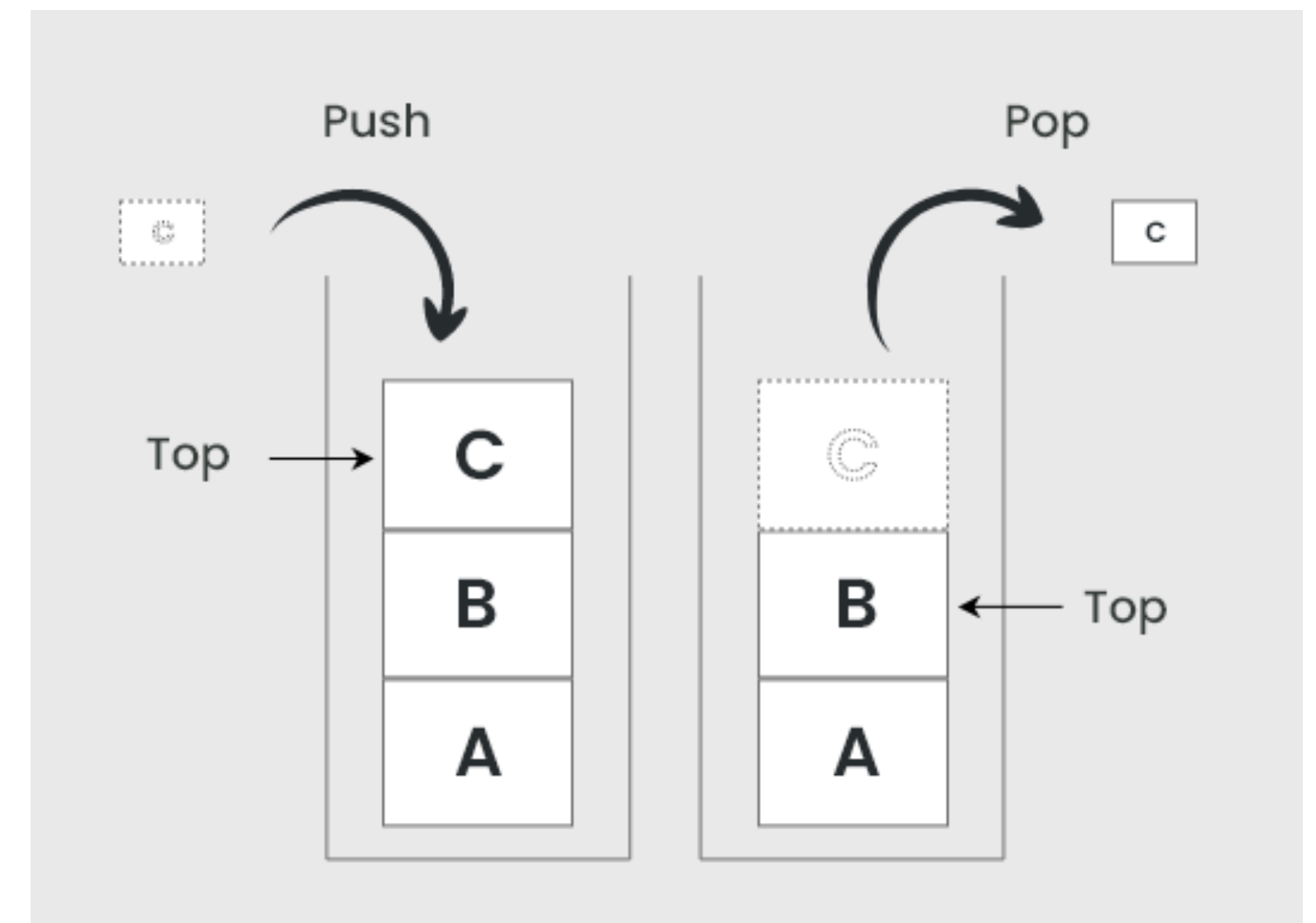


스택(Stack) : 쌓아놓은 더미



스택(Stack)이란?

- 후입선출(LIFO : Last In First Out)
- 가장 최근에 들어온 데이터가 가장 먼저 나감
- 이로 인해 출력 순서가 입력 순서의 역순일 때 자주 사용하는 자료구조





스택의 연산들

- **Push**

- 데이터를 스택의 가장 위에 추가

- **Pop**

- 스택의 가장 위에 있는 요소(element)를 제거

- **Top**

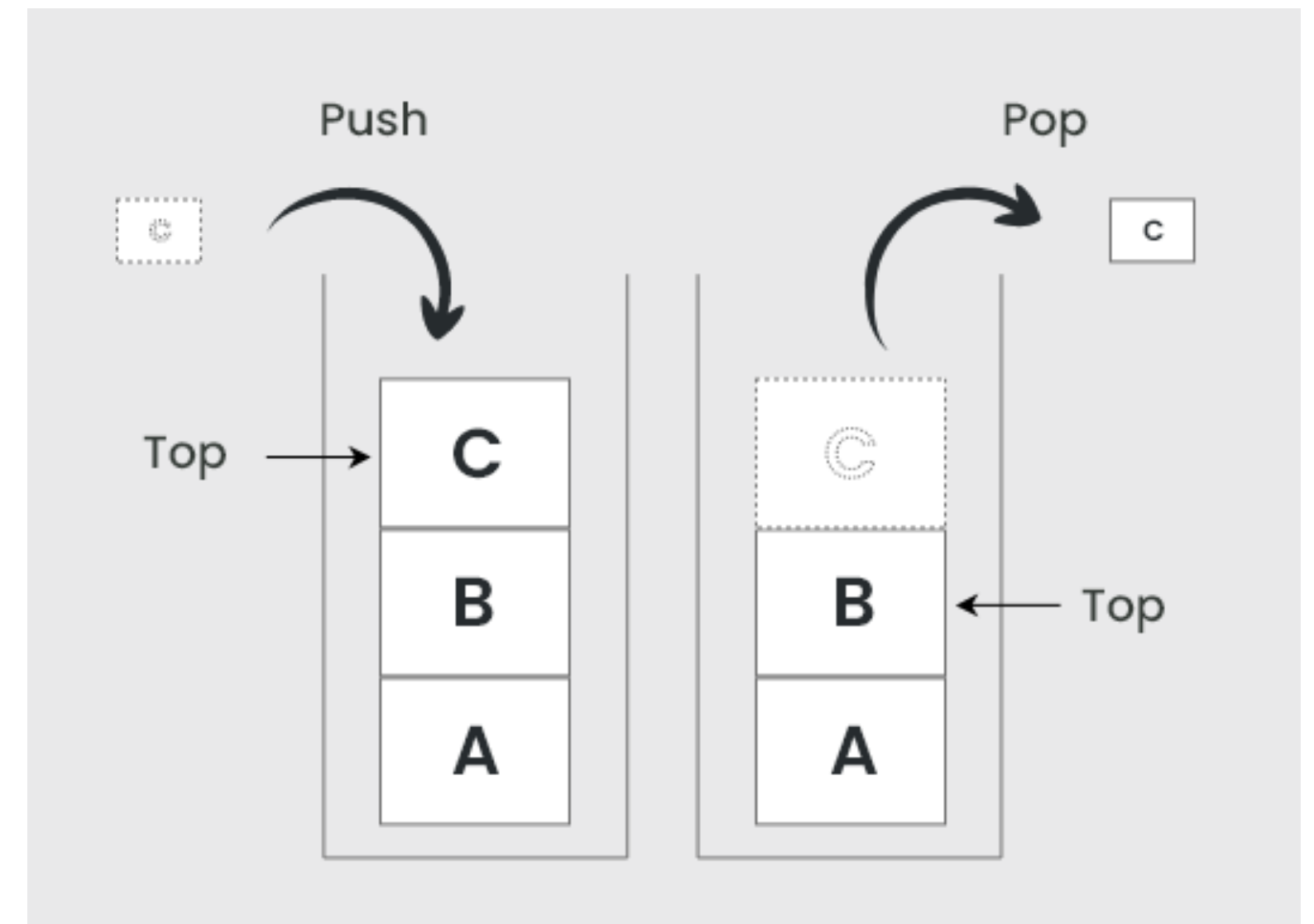
- 스택의 가장 위에 있는 요소를 반환

- **Empty**

- 스택이 비어 있는지 확인

- **Size**

- 스택에 있는 요소의 개수를 반환



Python

Deque를 활용하여 쉽게 구현 가능

Push

데이터를 스택의 가장 위에 추가

Pop

스택의 가장 위에 있는 요소(element)를 제거

Top

- 스택의 가장 위에 있는 요소를 반환
- 언어마다 사용하는 메소드가 다름

```
from collections import deque

stack = deque()

stack.append(1) # stack의 맨 위에 1 추가
stack.append(2) # stack의 맨 위에 2 추가

stack[-1] # 스택의 맨 위에 있는 요소(2) 반환
stack.pop() # 스택의 맨 위에 있는 요소(2)
           제거 및 반환
```



Empty

스택이 비어 있는지 확인

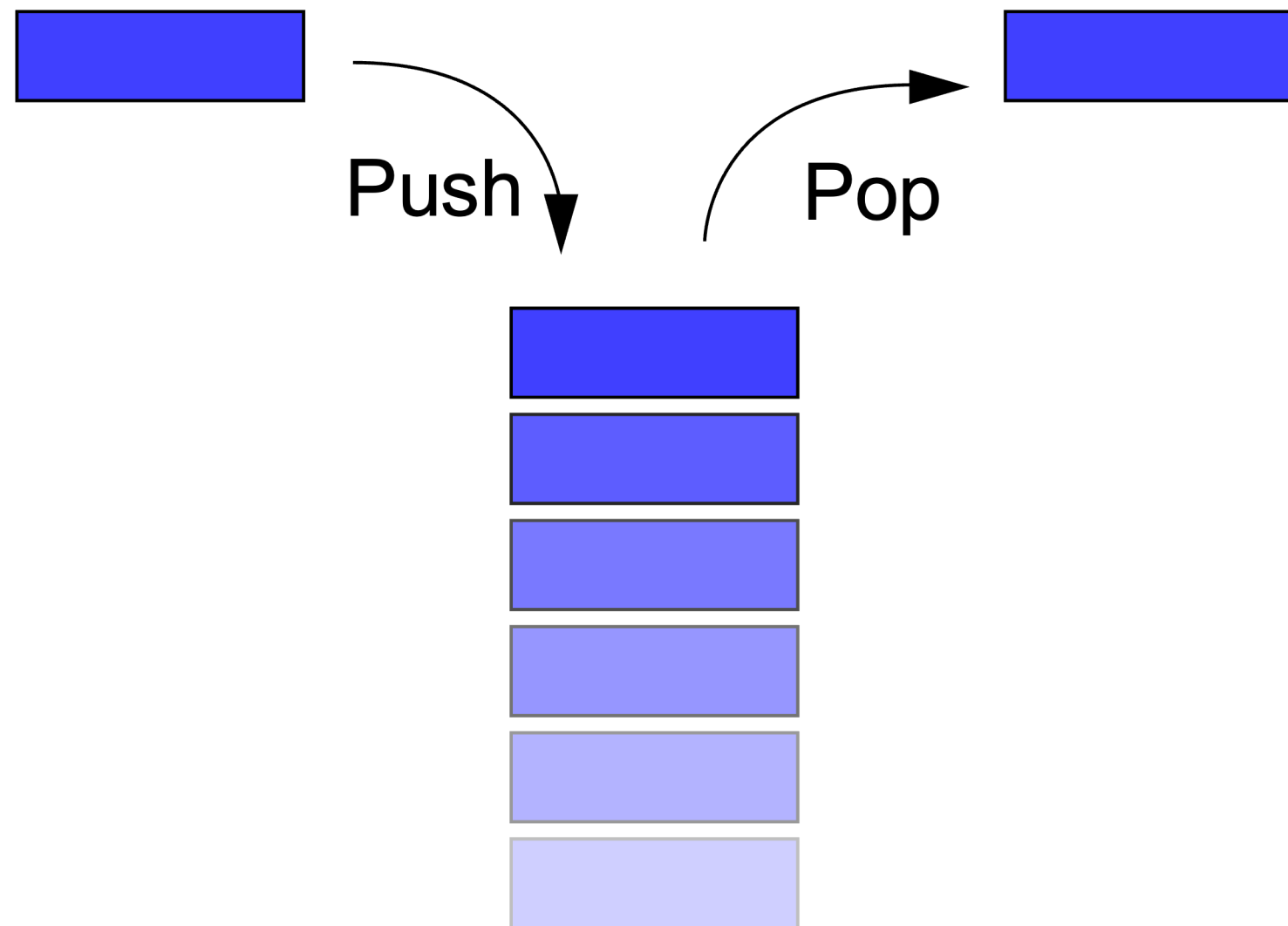
Size

스택에 있는 요소의 개수를 반환

```
if stack:
    print("스택이 비어 있지 않습니다")
else:
    print("스택이 비어 있습니다.")

print(len(stack))
```

올바른 괄호 문자열 - 스택(Stack)



정수형 변수 i 를 0부터 s 의 길이까지 반복:
만약 $s[i]$ 가 '('이라면:
 스택 st 에 '('를 push
그렇지 않으면, 만약 $s[i]$ 가 ')'이라면:
 만약 스택 st 가 비어있다면:
 "NO"를 출력하고, 프로그램 종료
 그렇지 않으면: 스택 st 에서 요소를 pop

만약 스택 st 가 비어있다면:
 "YES"를 출력
그렇지 않으면:
 "NO"를 출력

더 쉬운 방법은 없을까?

올바른 괄호 문자열 - 치환

- '('를 1, ')'를 -1로 치환
- 문자열 S 를 전부 순회하며 합 계산
- 중간에 합이 음수가 되거나,
계산이 모두 끝나고 합이 0이 되지 않으면
올바른 괄호 문자열이 아님
- 마지막에 합이 0이 아닌 것은
스택이 비어 있지 않는 것과 같은 의미

$s1 = "(())"$

i	0	1	2	3
s1[i]	1	1	-1	-1
sum	1	2	1	0

$s1$ 은 올바른 괄호 문자열이다.

올바른 괄호 문자열 - 치환

$s2 = ")()("$

i	0	1	2	3
s2[i]	-1	1	1	-1
sum	-1	0	1	0

중간에 sum이 -1이 되기 때문에,
올바른 괄호 문자열이 아니다.

$s3 = "()(("$

i	0	1	2	3
s3[i]	1	-1	1	1
sum	1	0	1	2

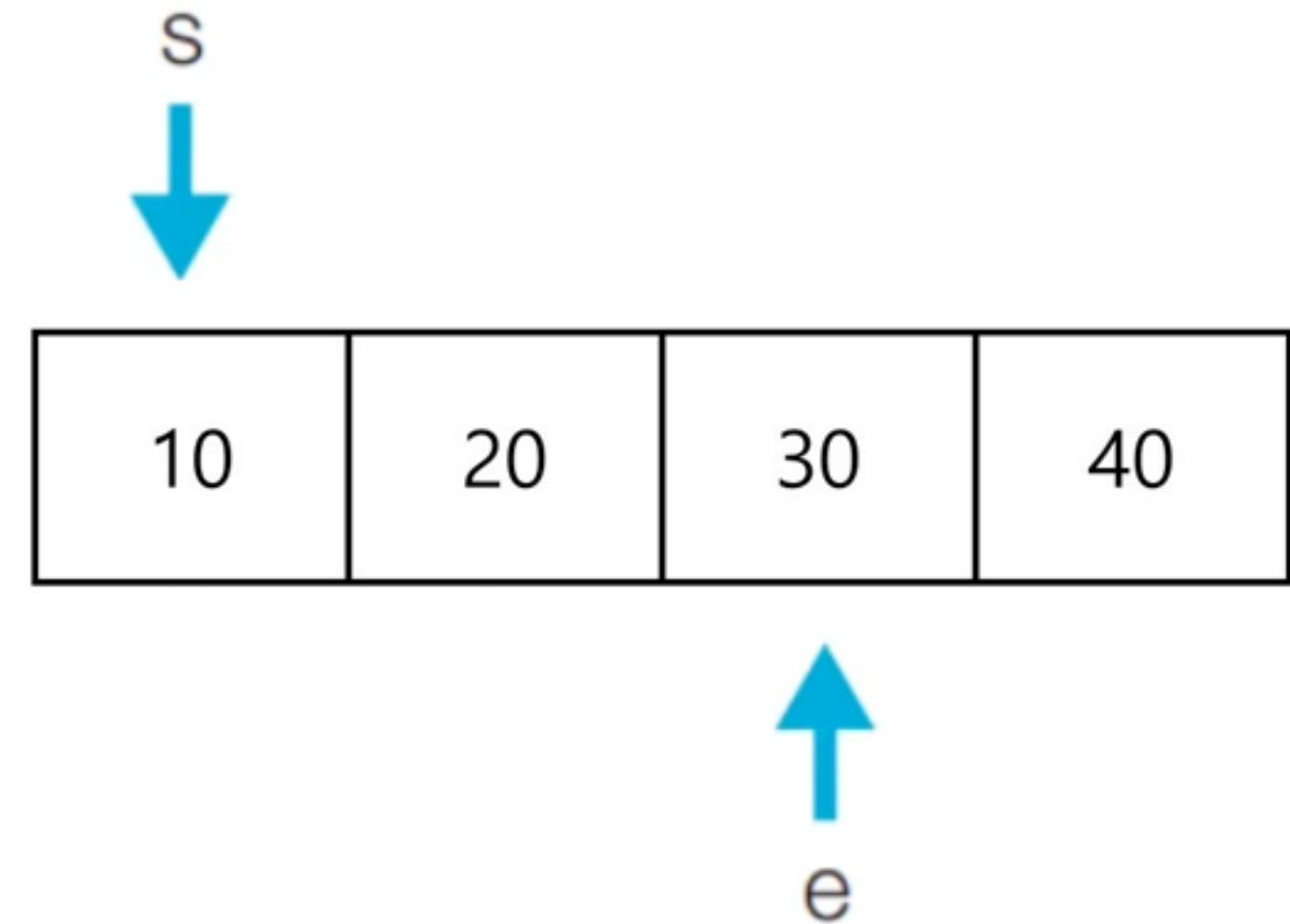
마지막에 sum의 합이 0이 아니므로,
올바른 괄호 문자열이 아니다.

투 포인터



투 포인터 (Two Pointer)

- 이름 그대로 **포인터 2개**를 사용해 두 점의 위치를 기록해가며 문제를 해결하는 기법(technique)
- 1차원 배열을 순차적으로 접근해야 할 때 사용
- 두 포인터의 이름은 다양하게 불림
 - (s, e) , (st, en) , $(start, end)$, (l, r) , $(left, right)$
- 시간 복잡도는 일반적으로 $O(N)$





투 포인터 (Two Pointer) 예시

크기가 N 인 배열 A 가 있다.

M 의 값이 주어질 때, A 배열의 i 번째 수부터 j 번째 수까지의 합($A[i] + A[i + 1] + \dots + A[j]$)이 M 이 되는 경우의 수를 구하여라.

$$1 \leq i \leq j \leq N$$

단, 배열의 인덱스는 1부터 시작한다고 가정한다.

($A = [A[1], A[2], \dots, A[N]]$)

$A = [10, 20, 30, 40, 10, 20, 30, 30, 40], M = 30$ 이라면?

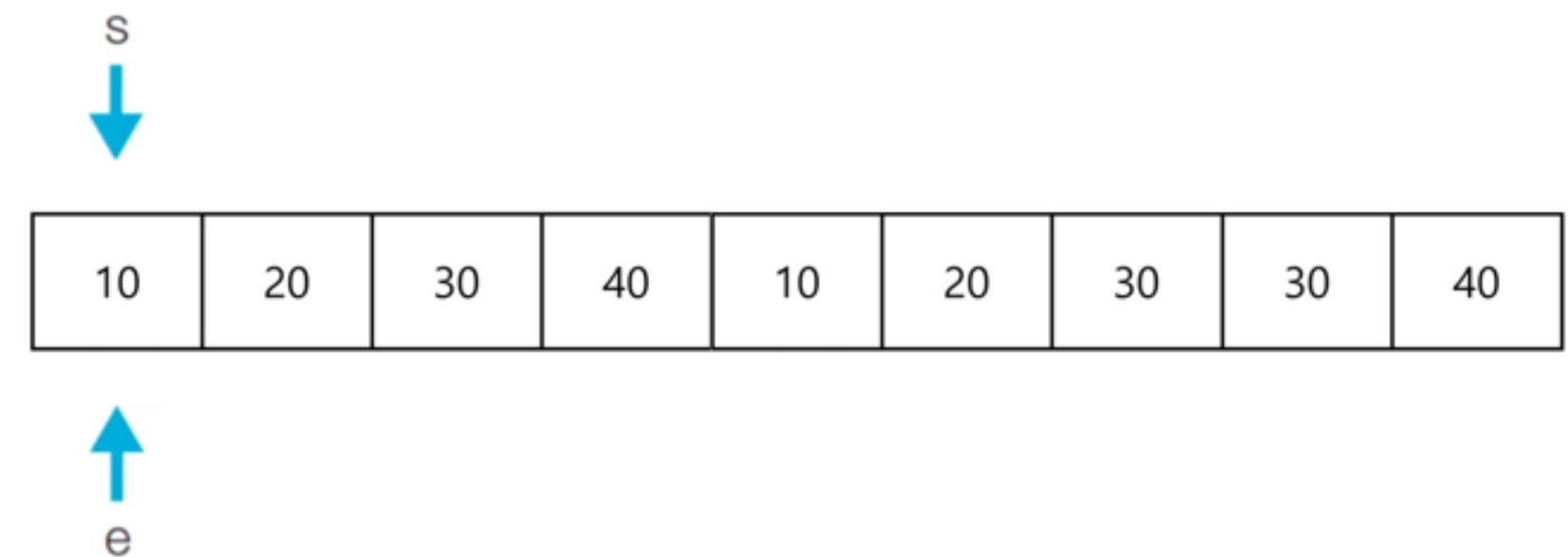
모든 경우의 수를 구한다면 $\rightarrow O(N^2)$

진행 방식

(부분합 $< M$) $\rightarrow e$ 증가

(부분합 $\geq M$) $\rightarrow s$ 증가

(부분합 $= M$) $\rightarrow cnt$ 증가

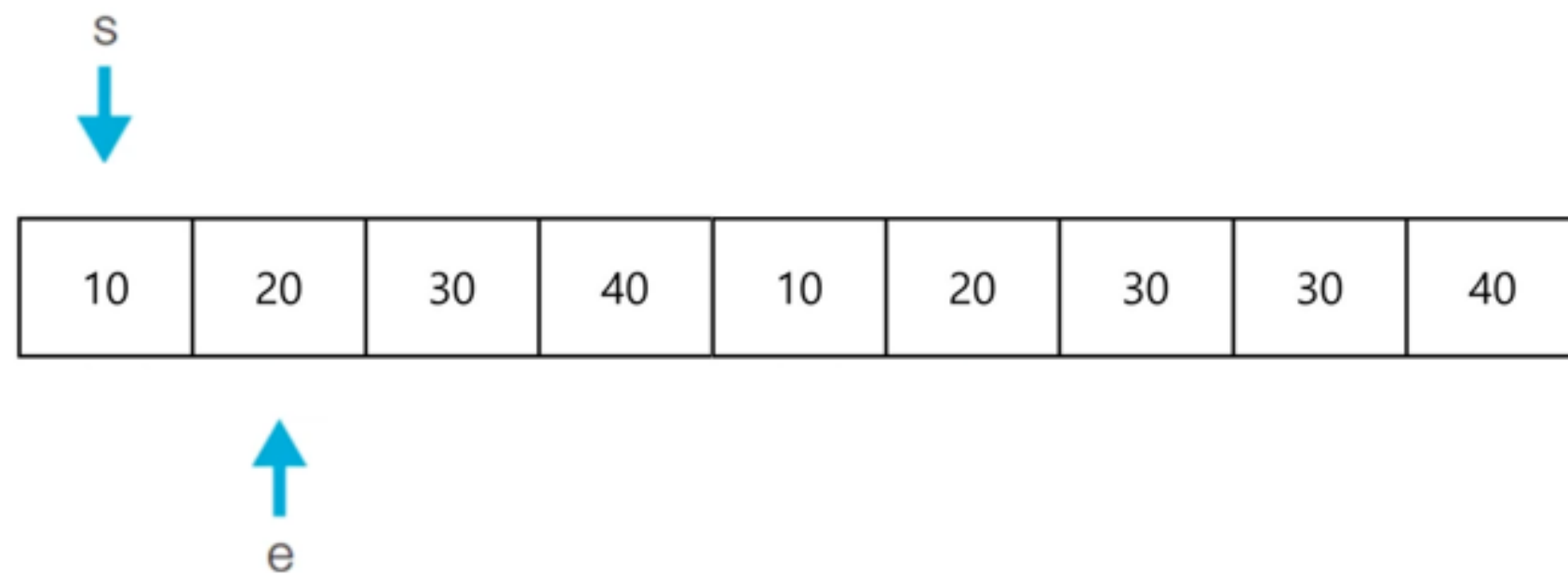


처음에는 두 포인터가 첫 번째 인덱스를 가리키도록 함
 M 이 되는 경우의 수를 cnt 라고 가정(초기 $cnt = 0$)

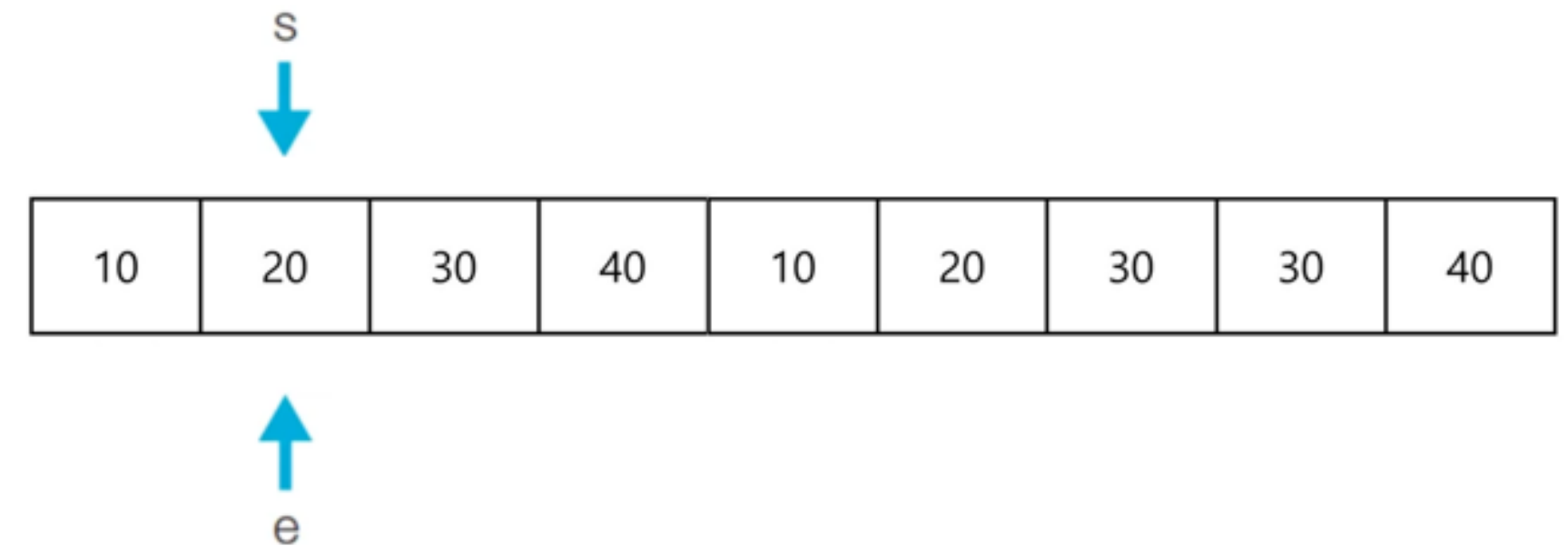
현재 부분합($s \sim e$)이 10 ($< M$) $\rightarrow$ 포인터 e 를 증가



투 포인터 (Two Pointer) 예시



부분합 : 30 (= M) $\rightarrow$ *cnt* 증가
부분합 $\geq$ M이므로 이번에는 *s* 증가

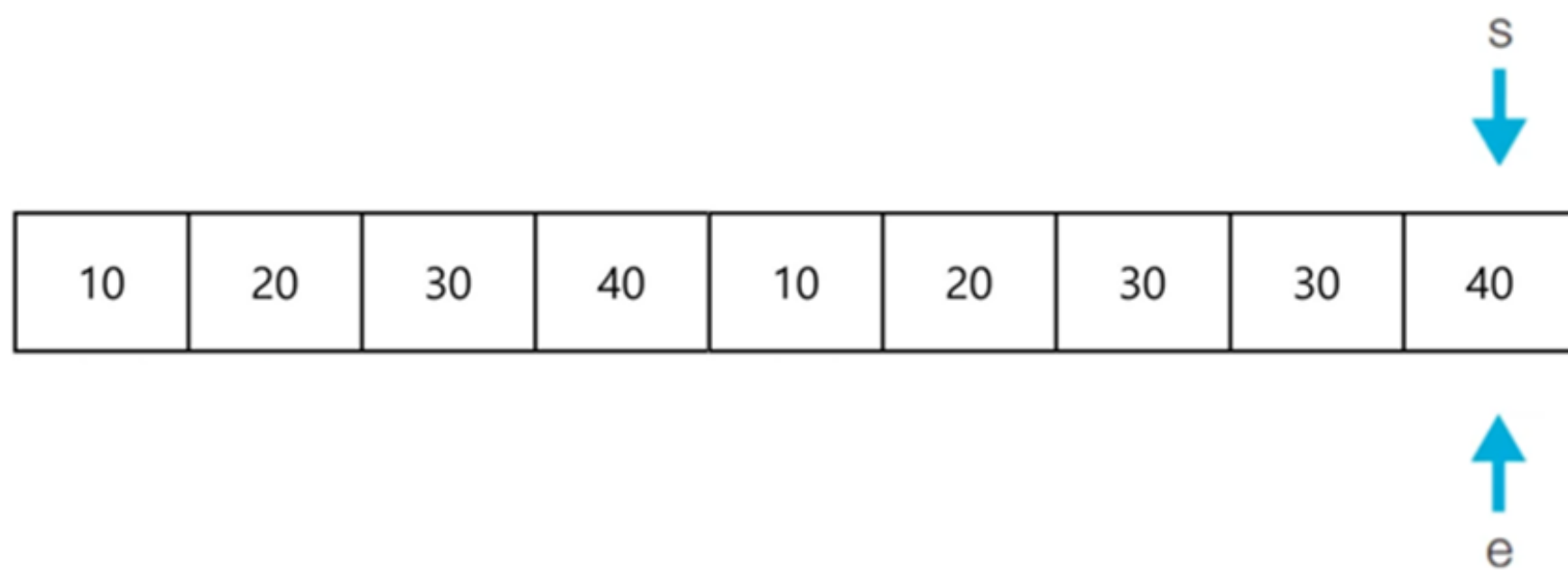


부분합 : 20
부분합 $<$ M이므로 이번에는 *e* 증가

왼쪽 포인터가 증가하면 합 감소, 오른쪽 포인터가 증가하면 합 증가



투 포인터 (Two Pointer) 예시



종료 상태

진행 방식

(부분합 < M) → e 증가

(부분합 ≥ M) → s 증가

(부분합 = M) → cnt 증가

슬라이딩 윈도우



슬라이딩 윈도우 (Sliding Window)

- 슬라이딩 윈도우 → 창문을 밀대로 닦는다
- 고정 사이즈의 윈도우(Window)가 이동하면서, 윈도우 내에 있는 데이터를 이용해 문제를 풀이하는 기법(technique)
- 투 포인터와 비슷하지만, 범위가 고정되어 있을 때 사용한다는 점에서 차이가 있음(두 포인터가 같이 이동)
- 시간 복잡도는 일반적으로 $O(N)$



슬라이딩 윈도우 (Sliding Window) 예시

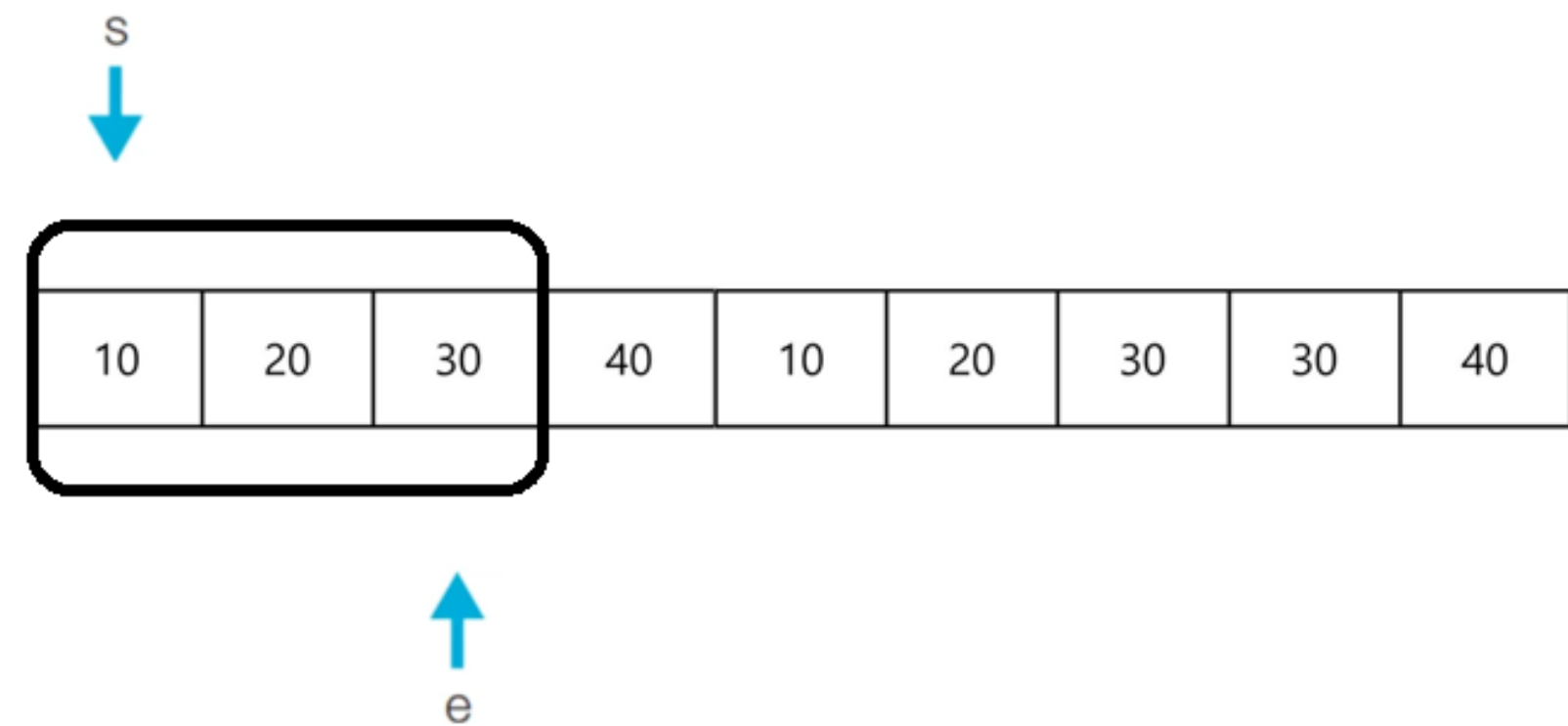
크기가 N 인 배열 A 가 있다.

M 의 값이 주어질 때, A 배열에서 서로 연속된 M 개의 값을 골라, 그 수들의 합을 구하려고 할 때, 합의 최댓값을 구하여라.

단, 배열의 인덱스는 1부터 시작한다고 가정한다.

($A = [A[1], A[2], \dots, A[N]]$)

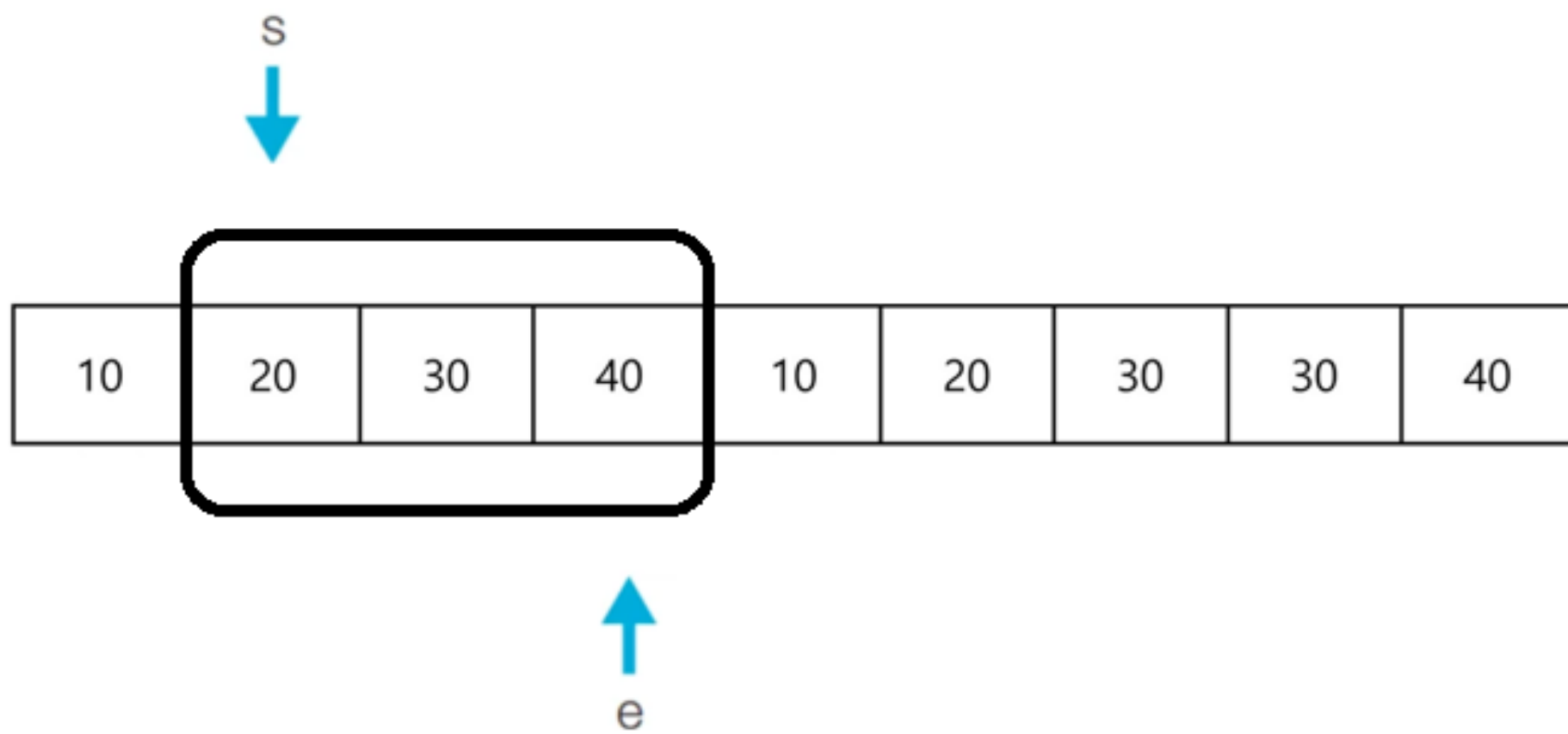
예를 들어, $A = [1, 2, 3, 4, 5]$ 이고, M 의 값이 3이면, 고른 수들의 위치가 연속되도록 $[1, 2, 3]$, $[2, 3, 4]$, $[3, 4, 5]$ 와 같이 골라야 한다.



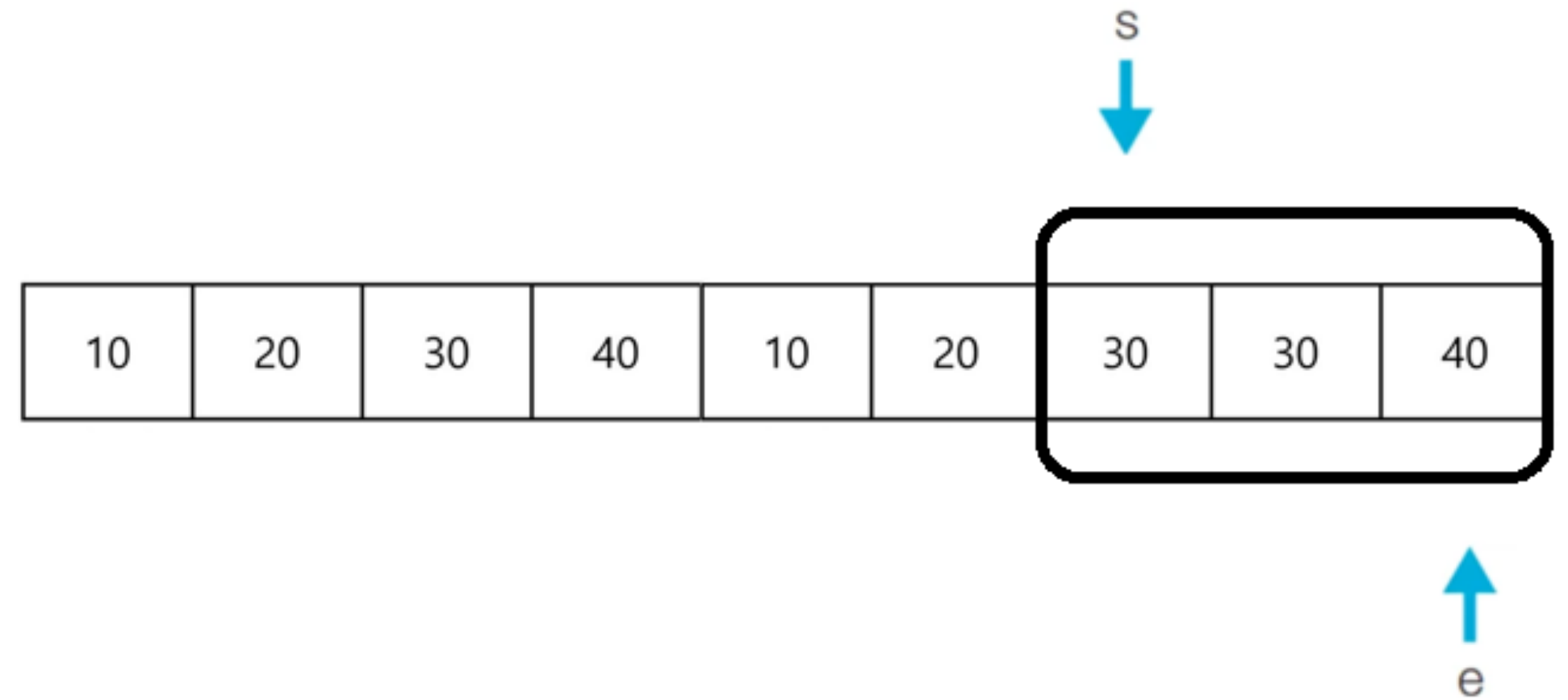
초기 s 는 0, e 는 $s + m$ 의 값을 가지게 됨

$A = [-22, 87, 62, 71, -27, -62, -72, 68, 82, 4]$
 $M = 3$ 이라면?

슬라이딩 윈도우 (Sliding Window) 예시



이후부터 s 와 e 의 값을 계속 1씩 증가



종료 상태